TEMA 1

INTRODUCCIÓN AL PARADIGMA ORIENTADO A OBJETOS

Cristina Cachero

Pedro J. Ponce de León

(1 Sesión)

Versión 0.6



Depto. Lenguajes y Sistemas Informáticos

Universitat d'Alacant
Universidad de Alicante

Indice



- **El progreso de la abstracción**
  - Definición de la abstracción
  - Principales paradigmas de programación
  - Lenguajes de programación y niveles de abstracción
  - Mecanismos de abstracción en los lenguajes de programación

- El paradigma orientado a objetos
  - Lenguajes orientados a objetos (LOO). Características básicas
  - LOO: Características opcionales
  - Historia de los LOO
  - Metas de la programación orientada a objetos (POO)

El progreso de la abstracción
Definición



- **Abstracción**
  - *Supresión intencionada (u ocultación) de algunos detalles de un proceso o artefacto, con el fin de destacar más claramente otros aspectos, detalles o estructuras*.

- Es una técnica para crear, comprender o manejar sistemas complejos.
  - En cada nivel de detalle cierta información se muestra y cierta información se omite.
  - Ejemplo: Diferentes escalas en mapas.

- Mediante la abstracción creamos **MODELOS** de la realidad.

- Se aplica a todos los ámbitos, no sólo a la Informática.

El progreso de la abstracción
Principales paradigmas



- **PARADIGMA***:*
  - Forma de entender y representar la realidad.
  - Conjunto de teorías, estándares y métodos que, juntos, representan un modo de organizar el pensamiento, es decir, el modo de 'ver' el mundo.

- Cada nuevo paradigma responde a una necesidad real de nuevos modos de afrontar problemas.

- Principales **paradigmas de programación**:

  - Paradigma *Funcional*: El lenguaje describe procesos
    - Lisp y sus dialectos (e.g. Scheme), Haskell, ML
  - Paradigma *Lógico*
    - Prolog
  - Paradigma *Imperativo* (o procedural)
    - C, Pascal
  - Paradigma *Orientado a Objetos*
    - Java, C++, Smalltalk, …

El progreso de la abstracción

Lenguajes de programación y niveles de abstracción



- Los lenguajes de programación proporcionan abstracciones

Espacio del problema

Lenguajes Orientados a Objetos (LOO)

LOO Puros

Smalltalk, Eiffel

Espacio de la solución

Lenguajes Ensamblador

Lenguajes Imperativos (C, Fortran, BASIC)

Lenguajes Específicos (LISP, PROLOG)

LOO Híbridos (Multiparadigma)

C++, Object Pascal, Java,…

El progreso de la abstracción

Mecanismos de abstracción en los lenguajes de programación



- Los diferentes niveles de abstracción ofertados por un lenguaje, desde los que se puede abordar una aplicación, dependen de los mecanismos proporcionados por el lenguaje elegido:
  - Ensamblador
  - Procedimientos
  - Módulos

  } **Perspectiva funcional**

  - Paquetes
  - Tipos abstractos de datos (TAD)

  } **Perspectiva de datos**

  - Objetos
    - TAD
    - + paso de mensajes
    - + herencia
    - + polimorfismo

  } **Perspectiva de servicios**

El progreso de la abstracción

Mecanismos de abstracción en los lenguajes de programación

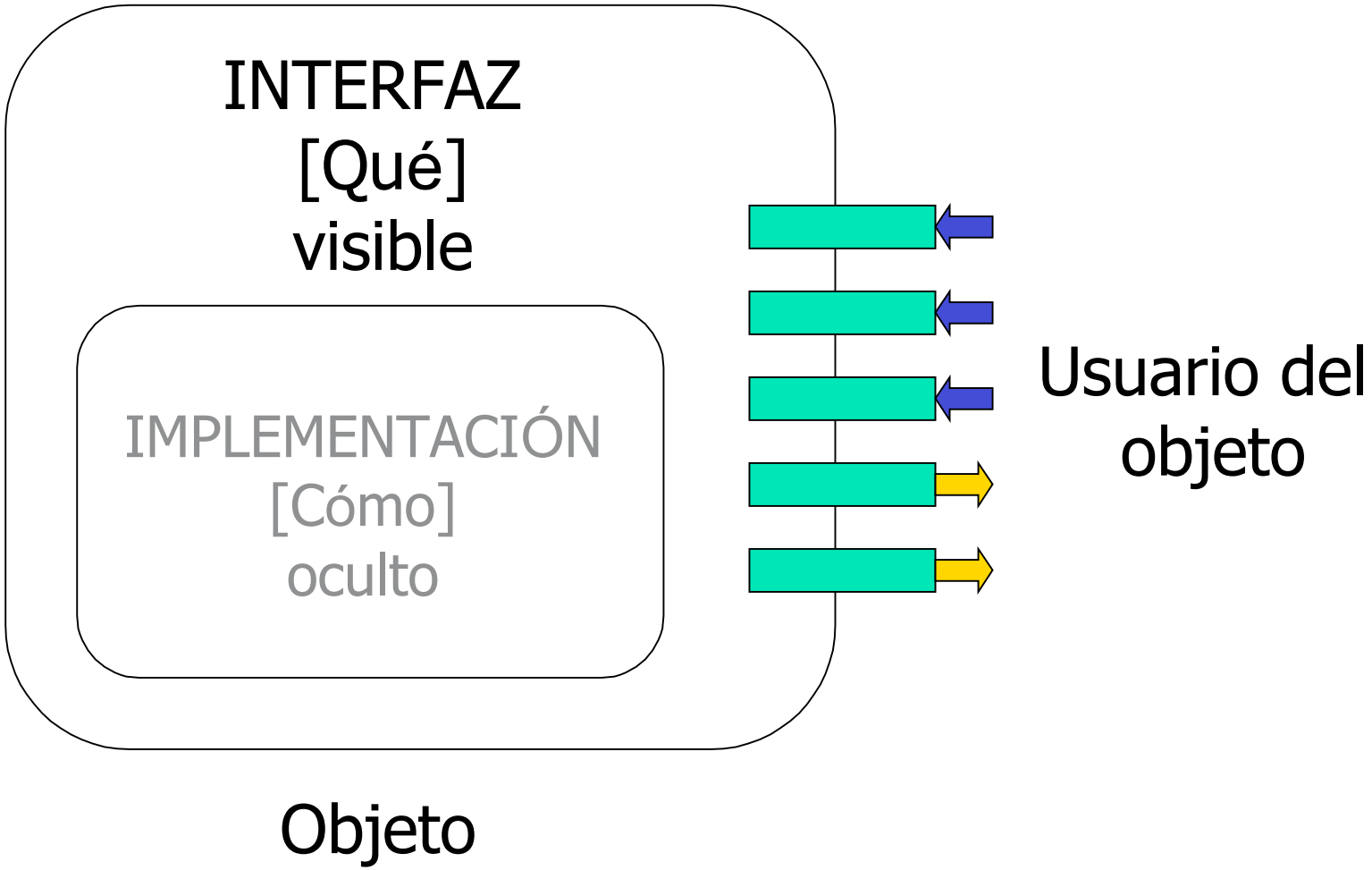


- **OCULTACIÓN DE INFORMACIÓN:**
  - Omisión intencionada de detalles de implementación tras una interfaz simple.

- Cuando además existe una división estricta entre la vista interna de un componente (objeto) y su vista externa hablamos de **ENCAPSULACIÓN**.
  - Estas dos vistas son:
    - **INTERFAZ**: QUÉ sabe hacer el objeto. Vista externa
    - **IMPLEMENTACIÓN**: CÓMO lo hace. Vista interna
  - Favorece la intercambiabilidad.
  - Favorece la comunicación entre miembros del equipo de desarrollo y la interconexión de los artefactos resultantes del trabajo de cada miembro.

El progreso de la abstracción

Mecanismos de abstracción en los lenguajes de programación



Indice



- El progreso de la abstracción
  - Definición de la abstracción
  - Principales paradigmas de programación
  - Lenguajes de programación y niveles de abstracción
  - Mecanismos de abstracción en los lenguajes de programación

- **El paradigma orientado a objetos**
  - Características básicas de los lenguajes orientados a objetos (LOO).
  - LOO: Características opcionales
  - Historia de los LOO
  - Metas de la programación orientada a objetos (POO)

El paradigma orientado a objetos



- Metodología de desarrollo de aplicaciones en la cual éstas se organizan como colecciones cooperativas de **objetos,** cada uno de los cuales representan una **instancia** de alguna **clase**, y cuyas clases son todas miembros de una jerarquía de clases unidas mediante relaciones de **herencia**. (Grady Booch)

- Cambia…
  - El modo de organización del programa:
    - En clases (datos+operaciones sobre datos).
  - El concepto de ejecución de programa
    - Paso de mensajes

- No basta con utilizar un lenguaje OO para programar orientado a objetos. Para eso hay que seguir un paradigma de programación OO.

El paradigma orientado a objetos



- Cambia el concepto de dato (*pasivo*) por el de **objeto** (*activo*).
  - Cada objeto es una especie de máquina funcional cerrada con unos pocos interruptores y diales bien definidos.

- Favorece la claridad y la estructuración: facilitan la auto-documentación de programas.
  - Los mecanismos de encapsulación facilitan la comprensión del programa y permite, por ejemplo, la generación automática de documentación.

El paradigma orientado a objetos
¿Por qué la POO es tan popular?



- POO se ha convertido durante las pasadas dos décadas en el paradigma de programación dominante, y en una herramienta para resolver la llamada *crisis del software*

- Motivos
  - POO escala muy bien, desde el problema más trivial hasta la tarea más compleja.
  - POO proporciona un modelo de abstracción que razona con técnicas que la gente usa para resolver problemas en sus vidas (metáforas)
    - *"Es más fácil enseñar Smalltalk a niños que a programadores"* (Kay 77)
  - Gran desarrollo de herramientas OO (IDEs, librerías,…) en todos los dominios.

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- Ejemplo: Supongamos que Luis quiere enviar flores a Alba, que vive en otra ciudad.
  - Luis va a la floristería más cercana, regentada por un florista llamado Pedro.
  - Luis le dice a Pedro qué tipo de flores enviar a Alba y la dirección de recepción.

- El mecanismo utilizado para resolver el problema es
  - Encontrar un **agente** apropiado (Pedro)
  - Enviarle un **mensaje** conteniendo la petición (envía flores a Alba).
  - Es la **responsabilidad** de Pedro satisfacer esa petición.
  - Para ello, es posible que Pedro disponga de algún **método** (algoritmo o conjunto de operaciones) para realizar la tarea.
- Luis no necesita (ni le interesa) conocer el método particular que Pedro utilizará para satisfacer la petición: esa *información está OCULTA*.

- Así, la solución del problema requiere de la cooperación de varios individuos para su solución.

- La definición de problemas en términos de responsabilidades incrementa el nivel de abstracción y permite una mayor independencia entre objetos.

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



Mundo estructurado en:

- Agentes y comunidades
- Mensajes y métodos
- Responsabilidades
- Objetos y clases
- Jerarquías de clases
- Enlace de métodos

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- **Agentes y comunidades**
  - Un programa OO se estructura como una comunidad de agentes que interaccionan (OBJETOS). Cada objeto juega un rol en la solución del problema. Cada objeto proporciona un servicio o realiza una acción que es posteriormente utilizada por otros miembros de la comunidad.

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- **Mensajes y métodos**
  - A un objeto se le envían mensajes para que realice una determinada acción.
  - El objeto selecciona un método apropiado para realizar dicha acción.
  - A este proceso se le denomina ***Paso de mensajes***
  - Ese mensaje puede ir acompañado de cualquier información adicional (parámetros) necesaria para atender la petición.
  - Sintáxis de un mensaje:

receptor.selector(argumentos)

```
aGame.displayCard(aCard,42,47)
```

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- **Mensajes y métodos**
  - Un mensaje se diferencia de un procedimiento/llamada a función en dos aspectos:
    - En un mensaje el receptor es conocido, mientras que en una llamada a procedimiento no existe receptor.
      - Por tanto un procedimiento tiene 0 o más argumentos, y un mensaje tiene 1 o más argumentos, bien sea explícito (CLOS, Python) o implícito (Java, C++)
    - La interpretación de un mismo mensaje puede variar en función del receptor del mismo.
      - Muchas veces este receptor no se conoce hasta el momento de la invocación (en tiempo de ejecución)
      - Por tanto un nombre de procedimiento/función se identifica 1:1 con el código a ejecutar, mientras que un mensaje no.

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- **Responsabilidades**
  - El comportamiento de cada objeto se describe en términos de responsabilidades -> incremento del nivel de abstracción
    - Mayor independencia entre los objetos, básico para la solución de problemas más complejos
  - **Protocolo**: Conjunto de responsabilidades de un objeto
  - POO vs Programación imperativa
    - Un programa tradicional opera SOBRE estructuras de datos.
    - Un programa OO solicita a las estructuras de datos (objetos) que realicen un servicio.

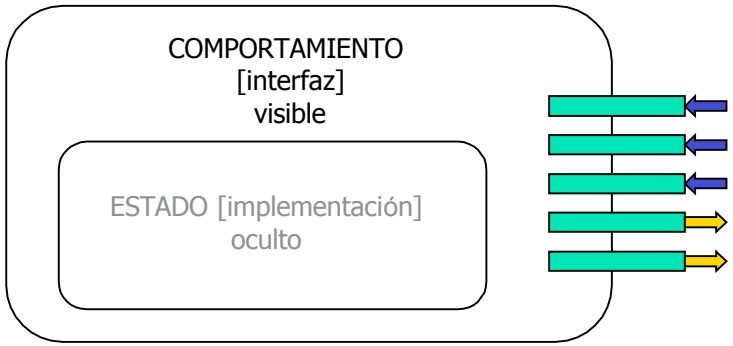
No preguntes lo que puedes hacer a tus estructuras de datos

Pregunta lo que tus estructuras de datos pueden hacer por ti

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- **Objetos y clases**
  - Un objeto es una **encapsulación** de un **estado** (valores de los datos) y **comportamiento** (operaciones).

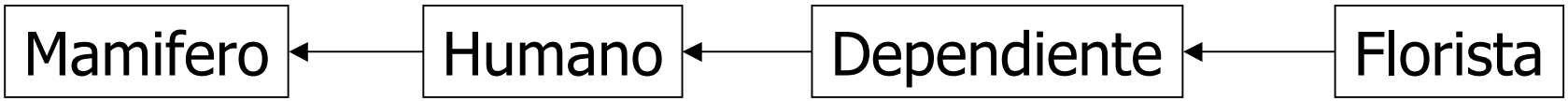


  - Los objetos se agrupan en categorías (**clases**).
    - Un objeto es una **instancia** de una clase.
    - El método invocado por un objeto en respuesta a un mensaje viene determinado por la clase del objeto receptor.

El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- **Jerarquías de clases: generalización (herencia)**
  - En la vida real, mucho conocimiento se organiza en términos de jerarquías. Este principio por el cual el conocimiento de una categoría más general es aplicable a una categoría más específica se denomina **generalización**, y su implementación en POO se denomina **herencia**.
    - Pedro, por ser florista, es un dependiente (sabe vender y cobrar)
    - Los dependientes normalmente son humanos (pueden hablar)
    - Los humanos son mamíferos (Pedro respira oxígeno…)
  - Las clases pueden ser organizadas en una estructura jerárquica de herencia. Una clase '*hijo*' heredará propiedades de una clase 'padre' más alta en la jerarquía (más general):



El paradigma orientado a objetos
Un nuevo modo de ver el mundo



- **Enlace de métodos**
  - Estático: en tiempo de compilación
  - Dinámico: en tiempo de ejecución

  - Incluso si el compilador no puede determinar qué método será invocado en tiempo de ejecución, en muchos lenguajes OO puede determinar si, llegado el momento, existirá algún método apropiado.

Indice



- El progreso de la abstracción
  - Definición de la abstracción
  - Principales paradigmas de programación
  - Lenguajes de programación y niveles de abstracción
  - Mecanismos de abstracción en los lenguajes de programación

- El paradigma orientado a objetos
  - **Características básicas de los lenguajes orientados a objetos**
  - LOO: Características opcionales
  - Historia de los LOO
  - Metas de la programación orientada a objetos (POO)

Características Básicas de un LOO



- Según Alan Kay (1993), son seis:
  1. Todo es un **objeto**
  2. Cada objeto tiene su propia memoria configurada a partir de otros objetos.
  3. Todo objeto es **instancia** de una **clase**
  4. Todos los objetos de la misma clase pueden recibir los mismos mensajes (realizar las mismas acciones). La clase es el lugar donde se define el **comportamiento** de los objetos y su estructura interna.
  5. Las clases se organizan en una estructura arbórea de raíz única, llamada **jerarquía de herencia**.
     1. La memoria y el comportamiento asociados con instancias de una clase están disponibles para cualquier clase asociada con un descendiente en esta estructura arbórea. P. ej: puesto que un círculo es una forma, un círculo debe siempre aceptar todos los mensajes destinados a una forma.
  6. Un programa es un conjunto de objetos que se comunican mediante el envío de mensajes.

Características opcionales de un LOO (1/3)



- **Polimorfismo**
  - Capacidad de una entidad de referenciar elementos de distinto tipo en distintos instantes
    - Enlace dinámico
- **Genericidad**
  - Definición de clases parametrizadas (*templates en C++*) que definen tipos genéricos.
    - Lista<T> : donde T puede ser cualquier tipo.
- **Gestión de Errores**
  - Tratamiento de condiciones de error mediante **excepciones**
- **Aserciones**
  - Expresiones que especifican que hace el software en lugar de cómo lo hace
    - **Precondiciones**:propiedades que deben ser satisfechas cada vez que se invoca una servicio
    - **Postcondiciones**:propiedades que deben ser satisfechas al finalizar la ejecución de un determinado servicio
    - **Invariantes**: aserciones que expresan restricciones para la consistencia global de sus instancias.

POO
Características opcionales de un LOO (2/3)



- **Tipado estático**
  - Es la imposición de un tipo a un objeto en tiempo de compilación
    - Se asegura en tiempo de compilación que un objeto entiende los mensajes que se le envían.
  - Evita errores en tiempo de ejecución

- **Recogida de basura** (*garbage collection*)
  - Permite eliminar automáticamente la memoria de aquellos objetos que ya no se utilizan.

- **Concurrencia**
  - Permite que diferentes objetos actúen al mismo tiempo, usando diferentes *threads* o hilos de control.

POO
Características opcionales de un LOO (3/3)



- **Persistencia**
  - Es la propiedad por la cual la existencia de un objeto trasciende en el tiempo.
    - Normalmente implica el uso de algún tipo de base de datos para almacenar objetos.

- **Reflexión**
  - Capacidad de un programa de manipular su propio estado, estructura y comportamiento.
    - En la programación tradicional, las instrucciones de un programa son 'ejecutadas' y sus datos son 'manipulados'.
    - Si vemos a las instrucciones como datos, también podemos manipularlas.

POO
Características opcionales de un LOO: conclusiones



- Lo ideal es que un lenguaje proporcione el mayor número posible de las características mencionadas
  - Orientación a objetos no es una condición booleana: un lenguaje puede ser 'más OO' que otro.
- No todo el mundo necesita todas las propiedades al mismo tiempo
- El criterio de 'ser más o menos OO' debe ser sólo uno de los factores que guíen la búsqueda de una solución software.
  - Este criterio debe tomarse en conjunción con otras consideraciones

Indice



- El progreso de la abstracción
  - Definición de la abstracción
  - Principales paradigmas de programación
  - Lenguajes de programación y niveles de abstracción
  - Mecanismos de abstracción en los lenguajes de programación

- El paradigma orientado a objetos
  - Características básicas de los lenguajes orientados a objetos (LOO).
  - LOO: Características opcionales
  - **Historia de los LOO**
  - Metas de la programación orientada a objetos (POO)

Historia de los L.O.O.: ***Simula***



- 1967. Norwegian Computer Center. Desarrollo de Simula para modelado y simulación de procesos industriales y científicos

- Es el primero en incorporar los conceptos de **clase, objeto, encapsulación, herencia y polimorfismo**

Historia de los L.O.O.: ***Smalltalk***



- Creado por Alan Kay en 1980 en Xerox PARC (Palo Alto Research Center) en su búsqueda de un lenguaje comprensible por personas sin formación informática
- Considerado lenguaje de programación "*puro*" (Todo es un objeto de una clase, y todas las clases heredan de una clases base).
- Énfasis en enlace dinámico
- Smalltalk afirmó los conceptos de **método** (acciones realizadas por un objeto) y **paso de mensajes** (medio para activar métodos)
- Descrito en la revista Byte en Agosto 1981

Historia de los L.O.O.: **Los 80'**



- Primera conferencia internacional sobre Lenguajes de Programación Orientados a Objetos (OOPSLA, 1986)

- En ese momento ya existen docenas de lenguajes O.O. : Eiffel, Actor, Objetive C, Object Pascal ...

- Muchos son extensiones a lenguajes existentes:
  - Extensiones a Smalltalk: **Smalltalk/V**
  - Extensiones OO a lenguajes convencionales: **Objetive C, C++, Java**
  - Extensiones a Pascal: **Object Pascal, Delphi**
  - Extensiones a LISP: **CLOS**

Historia de los L.O.O.: **Eiffel**



- Basado en Simula, también es considerado lenguaje orientado a objetos "*puro*".
- Lenguaje fuertemente tipado
- Añade construcciones para la verificación automática de programas (aserciones)

Historia de los L.O.O.: **C++**



- Desarrollado por Bjarne Stroustrup en los laboratorios Bell como una extensión de C.
- Aproximadamente contemporáneo a Smalltalk.
- Gran éxito comercial (1986->) por su compatibilidad con C: facilitó la aceptación de los principios OO por parte de programadores tradicionales.

Historia de los L.O.O.: **Actualidad**



- A partir de los 90’ proliferan con gran éxito la tecnología y lenguajes OO.
- Los más implantados en la actualidad son **Java**, **C++** y **PHP** (lista TIOBE)
- **C#, Python, Ruby, Delphi** son otros lenguajes OO muy utilizados
- Híbridos (OO, procedimental): PHP, C++, Visual Basic, Javascript
- Otros LOO: Objective-C, D, ActionScript

Indice



- El progreso de la abstracción
  - Definición de la abstracción
  - Principales paradigmas de programación
  - Lenguajes de programación y niveles de abstracción
  - Mecanismos de abstracción en los lenguajes de programación

- El paradigma orientado a objetos
  - Características básicas de los lenguajes orientados a objetos (LOO).
  - LOO: Características opcionales
  - Historia de los LOO
  - **Metas de la programación orientada a objetos (POO)**

Metas de la P.O.O.
Parámetros de Calidad (Bertrand Meyer)



- La meta última del incremento de abstracción de la POO es
  - **MEJORAR LA CALIDAD DE LAS APLICACIONES**.
- Para medir la calidad. Meyer define unos parámetros de calidad:
  - **PARÁMETROS EXTRÍNSECOS**
  - **PARÁMETROS INTRÍNSECOS**

Metas de la P.O.O.
Principales parámetros Extrínsecos



- **Fiabilidad: corrección** + **robustez**:
  - **Corrección**: capacidad de los productos software para realizar con exactitud sus tareas, tal y como se definen en las especificaciones.
    - tipado estático, aserciones, manejo automático de memoria,…
  - **Robustez**: capacidad de los sistemas software de reaccionar apropiadamente ante condiciones excepcionales.
    - manejo disciplinado de las excepciones

- Corrección: Si un sistema no hace lo que se supone que debe hacer, de poco sirve todo lo demás.
- La robustez por otro lado complementa la corrección. La corrección tiene que ver con el comportamiento de un sistema en los casos previstos por su especificación. La robustez caracteriza lo que sucede fuera de tal especificación.

Metas de la P.O.O.
Principales parámetros Intrínsecos



- **Modularidad: extensibilidad** + **reutilización**:
  - **Extensibilidad**: facilidad de adaptar los productos de software a los cambios de especificación.
    - Simplicidad de diseño
    - Descentralización del diseño: componentes autocontenidos
  - **Reutilización**: Capacidad de los elementos software de servir para la construcción de muchas aplicaciones diferentes. Es garantía de corrección.
    - Los sistemas sw a menudo siguen patrones similares

- En definitiva: producir aplicaciones + fáciles de cambiar: **mantenibilidad**

Conclusiones



- Lo que conocemos por P.O.O. no es un conjunto de rasgos añadidos a los lenguajes de programación. Más bien es un **nuevo modo de organizar el pensamiento** acerca del modo de descomponer problemas y desarrollar soluciones de programación.
- La POO ve un programa como un conjunto de agentes débilmente acoplados (objetos). Cada objeto es responsable de un cjto de tareas. La computación se realiza gracias a la interacción de estos objetos. Por tanto, en cierto sentido, programar consiste en **simular un modelo del universo.**
- Un objeto es una **encapsulación** de un estado (valores de los datos) y comportamiento (operaciones). Así, un objeto es en muchos sentidos similar a un ordenador de propósito específico.
- El comportamiento de los objetos viene dictado por su **clase**. Todos los objetos son **instancias** de alguna clase. Todas las instancias de la misma clase se comportarán de un modo similar (invocarán el mismo método) en respuesta a una petición similar.
- La interpretación de un **mensaje** es decidida por el objeto, y puede diferir de una clase de objeto a otra.

Conclusiones



- Las clases pueden enlazarse unas a otras mediante la noción de jerarquías de **herencia**. En estas jerarquías, datos y comportamiento asociados con clases más altas en la jeraquía pueden ser accedidas y usadas por clases que descienden de ellas.
- El diseño de un programa OO es como organizar una **comunidad** de individuos. Cada miembro de la comunidad tiene ciertas **responsabilidades**. El cumplimiento de las metas de la comunidad como un todo viene a través del trabajo de cada miembro y de sus interacciones.
- Mediante la reducción de la interdependencia entre componentes software, la POO permite el desarrollo de sistemas sw **reutilizables**. Estos componentes pueden ser creados y testados como unidades independientes, aisladas de otras porciones de la aplicación software.

Conclusiones



- Los componentes reutilizables permiten al programador tratar con problemas a un **nivel de abstracción** superior. Podemos definir y manipular objetos simplemente en términos de mensajes, ignorando detalles de implementación. Este principio de '**ocultación de información'** ayuda en la comprensión y construcción de sistemas seguros. También favorece la *mantenibilidad* del sistema.
- Se ha comprobado que a las personas con ciertos conocimientos tradicionales sobre computación les resulta más difícil aprender los nuevos conceptos que aporta la P.O.O. que a aquéllos que no saben nada, ya que el modo de razonar a la hora de programar es una **metáfora** del modo de razonar en el mundo real.
- La P.O.O. está fuertemente ligada a la **Ingeniería del Software**, con el objetivo de conseguir aplicaciones de mayor calidad.

Bibliografía



- Cachero et. al.
  - ***Introducción a la programación orientada a Objetos***
    - Capítulo 1
- Timothy Budd. *An introduction to OO Programming. 3rd Edition*. Addison Wesley, 2002
  - Capítulos 1 y 2
- Bertrand Meyer. *Object Oriented Software Construction*
- *Thinking in C++ / Thinking in Java* (online)